Angular Assignment: SmartShop Portal

Objective

Create an Angular application called **SmartShop Portal** that demonstrates:

- Angular Routing
- Inter-component Communication
- API calls using DummyJSON
- RxJS usage
- Login-based personalized salutation
- Route protection using Auth Guard

Application Name

SmartShop Portal

Requirements

Build the following pages/components.

---

1. Login Component

Create a login form with:

- Username
- Password

Use the DummyJSON login API:

```
POST https://dummyjson.com/auth/login
```

Expected Functionality

- On successful login, store the logged-in user details in a shared service.
- After successful login, navigate to the dashboard.
- Display personalized salutation after login.

Example

```
Welcome, Ramu!
```

---

2. Dashboard Component

The Dashboard component should:

- Show the logged-in user's name using RxJS `BehaviorSubject`.
- Display a menu with links to:

```
Products
Product Details
Profile
```

---

3. Products Component

Fetch the product list from DummyJSON API:

```
GET https://dummyjson.com/products
```

Display the following product details:

- Product name
- Price
- Rating
- Thumbnail

Expected Functionality

- On clicking a product, navigate to the Product Details page using routing.

---

4. Product Details Component

Read the product ID from the route parameter.

Fetch single product details from DummyJSON API:

```
GET https://dummyjson.com/products/{id}
```

Display

- Complete product information
- Product image
- Price
- Description
- Rating
- Brand/category if available

---

5. Header Component

The Header component should:

- Be visible only after login.
- Show salutation dynamically.

Example

```
Hello, <logged-in-user-firstname>
```

Expected Functionality

- The salutation should update using RxJS when login happens.
- Include a Logout button.

---

6. Inter-Component Communication

Use a shared service to pass logged-in user data from:

```
Login Component → Header Component
Login Component → Dashboard Component
```

Requirement

Use RxJS:

```
BehaviorSubject
```

or

```
Subject
```

---

7. Routing

Configure the following routes:

```
/login
/dashboard
/products
/products/:id
/profile
```

---

8. Route Protection

Protect the following routes using an Auth Guard:

```
/dashboard
/products
/products/:id
/profile
```

Expected Functionality

- If the user is not logged in, redirect to `/login`.
- If the user is logged in, allow access to protected routes.

---

9. RxJS Requirement

Use RxJS in the application.

Mandatory Usage

- Use `BehaviorSubject` to maintain logged-in user state.
- Use `Observable` in components to subscribe/display user data.
- Use at least one of the following operators in API service methods:

```
tap
map
catchError
```

Expected Output

The completed Angular application should have:

- A working login page
- Personalized dashboard
- Dynamic header salutation
- Product listing page
- Product details page
- Protected routes
- Logout functionality
- RxJS-based shared user state

Suggested Folder Structure

```
src/app/
│
├── components/
│   ├── login/
│   ├── dashboard/
│   ├── products/
│   ├── product-details/
│   ├── profile/
│   └── header/
│
├── services/
│   ├── auth.service.ts
│   └── product.service.ts
│
├── guards/
│   └── auth.guard.ts
│
├── app-routing.module.ts
└── app.module.ts
```

Evaluation Criteria

Criteria	Marks
Login using DummyJSON API	15
Routing implementation	15
Auth Guard route protection	15
Product API integration	15
Product details using route parameter	10
RxJS BehaviorSubject implementation	15
Inter-component communication	10
UI presentation and code quality	5
Total	**100**

Submission Instructions

Submit the Angular project with:

- Complete source code
- Proper folder structure
- Screenshots of output
- Short README file explaining how to run the project

Run Commands

```
npm install
ng serve
```

Open the application in browser:

```
http://localhost:4200
```